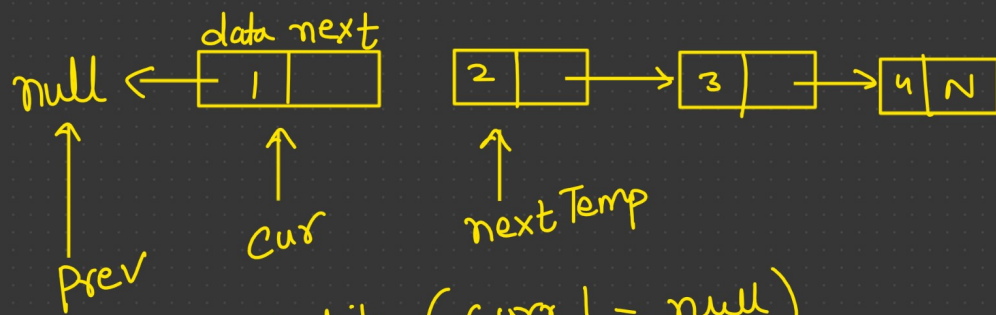
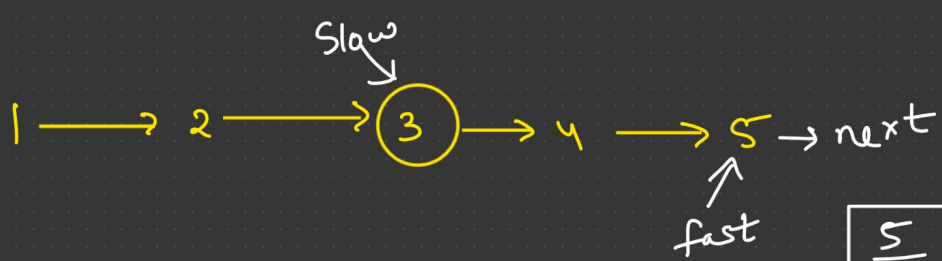




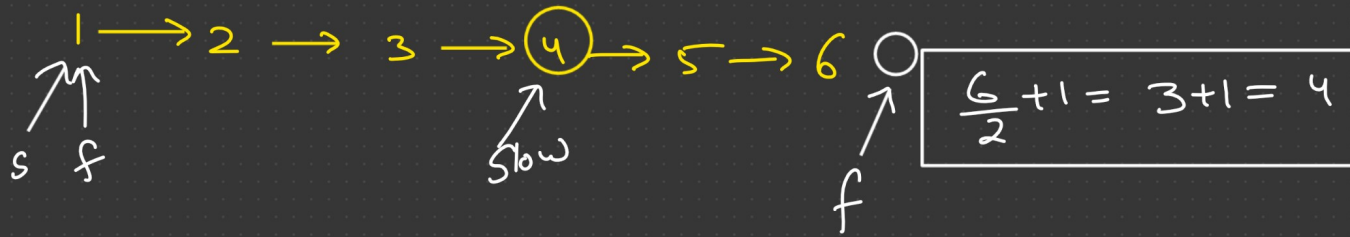
```

while (curr != null)
{
    nextTemp = cur->next
    cur->next = prev
    prev = cur
    cur = nextTemp
}

```



$$\frac{5}{2} + 1 = 2 + 1 = 3$$



$$\frac{6}{2} + 1 = 3 + 1 = 4$$

Node *slow = head;

Node *fast = head;

while (fast != null && fast->next != null)

{
 slow = slow->next;
 fast = fast->next->next;

}

return slow;

```

Node *temp = head;  int answer = 0;
while (temp != null)
{
    answer = (temp->data) + (answer*2)
}
    temp = temp->next;
return answer;

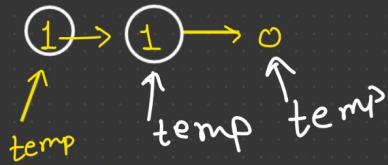
```

$$1 + 0 = \overset{\text{answer}}{\cancel{1}} \textcircled{3}$$

$$1 * 2 + 1$$

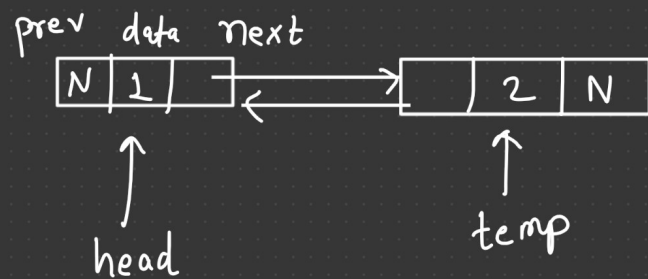
$$3 * 2 + 0$$

$$= 6$$



repeat $\rightarrow [(result) * 2 + \text{current value}]$

Doubly linked list



```
Node *temp = new Node();
```

```
temp->data = value;
```

```
head->next = temp;
```

```
temp->prev = head;
```

```

class node
{
    public:
        int data;
        node * next;
        node * prev;

```

```

    node()
    {
        next = null;
        prev = null;
    }
};

```

```

class DLL

```

```

{
    node * head = null;

    public:

```

```

    void insert_at_end(int value)

```

```

    {
        node * temp = new node();

```

```

        temp->data = value;

```

```

        node * t = head;

```

```

        if ( head == null )

```

```

        {
            head = temp;

```

```

        }

```

```

        else
        {

```

```

            while ( t->next != Null )

```

```

            {
                t = t->next;

```

```

            }

```

last
pe jar

```
t → next = temp;
temp → prev = t;
}
```

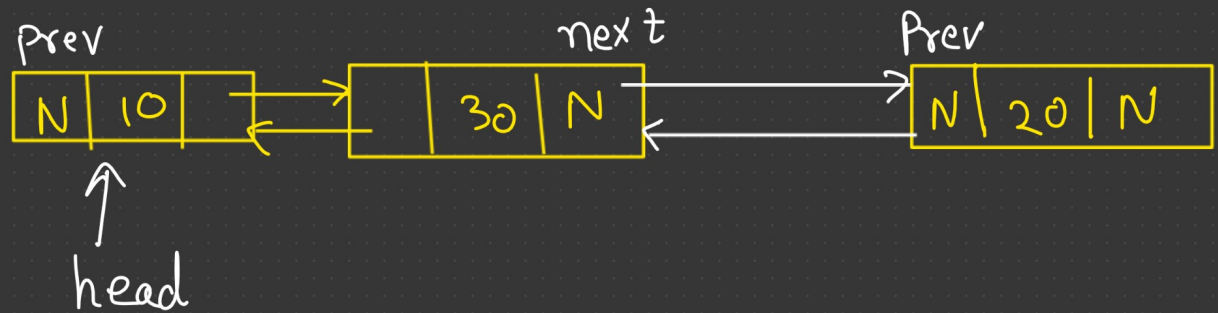
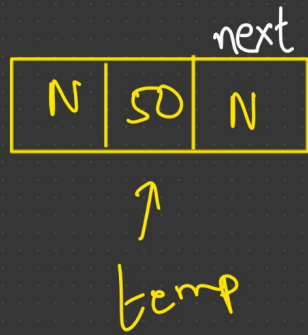
} update
2 pointers

```
}
```

```
void insert_at_begin(int value)
```

```
{
    node * temp = new node();
    temp → data = value;
    if ( head == null )
    {
        head = temp;
    }
    temp → next = head;
    head → prev = temp;
    head = temp;
}
```

} create node



```

Void deletion_at_begin()
{
    node * temp = head;
    head = head → next;
    head → prev = null;
    delete (temp);
}

```

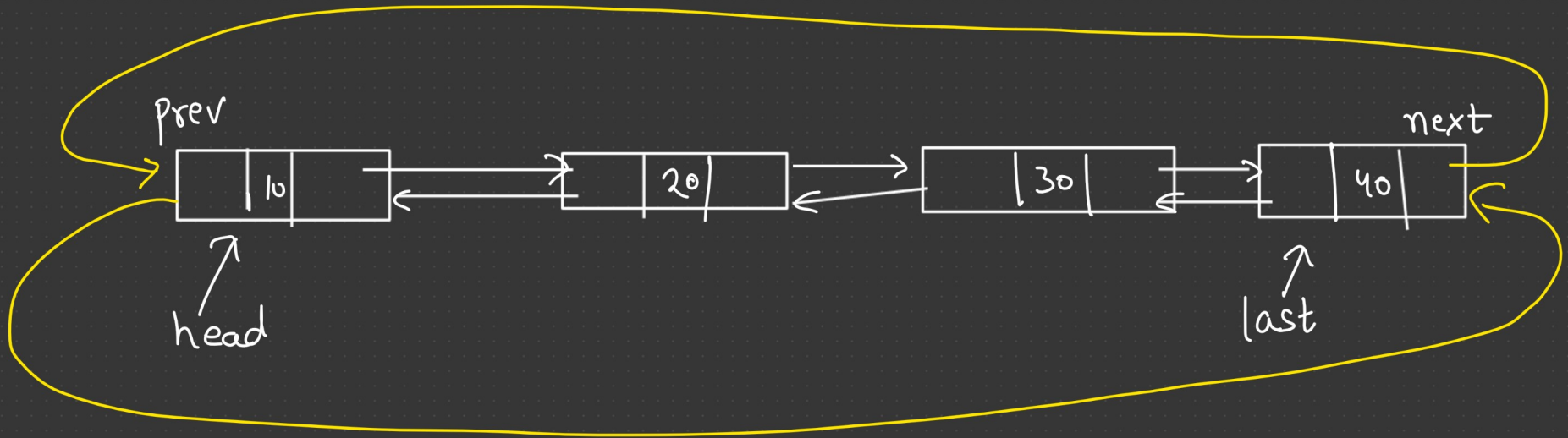
for circular LL

```

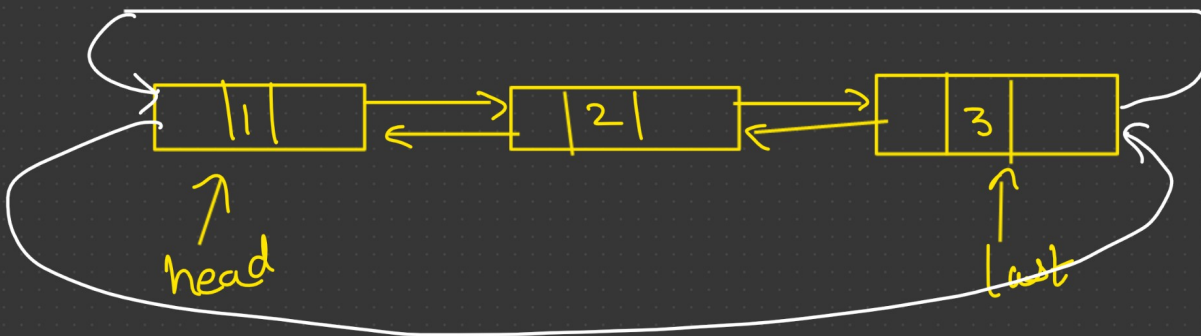
[ head → prev = last ]
[ last → next = head; ]

```


Circular linked list:-



node *last = head->prev;



last->next = head;
head->prev = last;

① Complete DLL code

② Complete CLL code

③ Leetcode;— a) Convert DLL to array
b) minimum pair removal to sort array.